

PR1: Backend — Gemini Ephemeral Token + Transcript Relay

Context

Adding voice-first taste discovery using Gemini Live API. The browser connects directly to Gemini via Web-Socket for speech-to-speech. Transcripts from voice conversations are sent to the backend, which processes them through the existing persona pipeline (Haiku signal detection -> Sonnet persona build -> CLIP embedding) to update recommendations.

PR1 delivers the backend: two new endpoints under `/api/voice/`.

Files to Create

`backend/app/schemas/voice.py`

Request/response models following CamelModel convention: - `VoiceTokenResponse(token: str, model: str)` - `TranscriptMessage(role: str, content: str)` - `TranscriptRequest(session_id: str, messages: list[TranscriptMessage])` - `TranscriptResponse(session_id: str, persona: Persona | None, recommendations: list[Recommendation])`

`backend/app/services/voice.py`

Gemini ephemeral token creation: - `create_ephemeral_token(settings)` -> tuple[str, str] — uses google-genai SDK - Late import of google.genai (only needed at call time) - Raises `ExternalServiceError("gemini", ...)` on failure - Uses `settings.gemini_live_model` and `settings.gemini_api_key`

`backend/app/routers/voice.py`

Two endpoints: - `POST /voice/token` — calls `create_ephemeral_token()`, returns token + model - DI: settings - `POST /voice/transcript` — runs transcript through persona pipeline, returns updated persona + recommendations - DI: settings, qdrant_client, anthropic_client - Flow: load session -> append messages -> `post_process_messages()` -> `build_persona()` (if new signals) -> `embed_persona()` -> `get_recommendations()` (if enough signals) -> save session -> return - Follows the `feedback.py` pattern of calling services directly

`backend/tests/test_voice_router.py`

Tests following existing `test_recommend_router.py` pattern: 1. Token endpoint returns token+model (mock `create_ephemeral_token`) 2. Token endpoint returns 502 when API key missing 3. Transcript runs full pipeline (monkeypatch all 4 stages) 4. Transcript skips persona build when no new signals 5. Transcript appends to existing session messages 6. Transcript returns empty recommendations when insufficient signals

Files to Modify

`backend/app/config.py`

Add: `gemini_live_model: str = "gemini-3.1-flash-live-preview"` (after `gemini_api_key`)

`backend/app/main.py`

- Import: add voice to from `app.routers` import `chat`, `feedback`, `recommend`, `session`, `voice`
- Register: `app.include_router(voice.router, prefix="/api")`

backend/pyproject.toml

Add "google-genai>=1.0.0" to dependencies

backend/.env.example

Add GEMINI_API_KEY= and GEMINI_LIVE_MODEL=gemini-3.1-flash-live-preview

Key Decisions

1. **Transcript endpoint is synchronous (not fire-and-forget)** — frontend needs persona + recommendations back immediately
2. **No get_gemini_client DI provider** — client only needed for token creation, not worth a shared dependency
3. **Late import of google.genai** — avoids import errors when package not installed in test env
4. **genai.Client may be sync** — wrap in `asyncio.to_thread()` if needed
5. **Router prefix /voice** — endpoints become `/api/voice/token` and `/api/voice/transcript`

Verification

1. `cd backend && pip install -e ".[dev]"` (install google-genai)
2. `ruff check app/ tests/` — lint passes
3. `mypy app/` — typecheck passes
4. `pytest tests/test_voice_router.py -v` — all tests pass
5. **Manual:** set GEMINI_API_KEY in .env, start server, `curl -X POST http://localhost:8000/api/voice/token -H 'Content-Type: application/json' -d '{"sessionId":"test"}'` — returns token